Computer Organization & Architecture

GATE CSE 2027 · complete formula, rule, method & trap sheet

98 reference boxes · **9** chapters ·

Number Systems & Computer Arithmetic | Instruction Set & Addressing Modes | Design of the Arithmetic Logic Unit (ALU) | Design of the Control Unit | Memory Hierarchy & Cache Mapping | I/O Interface: Interrupts & DMA | Instruction Pipelining & Hazards | Problem → Standard Method Playbook | Last-minute one-liners

1 Number Systems & Computer Arithmetic

Signed number representation Core

NOTES

A computer only has 0s and 1s. To represent negative numbers, we use one of three schemes on an **n-bit** word. GATE almost only cares about the third one.

REFERENCE TABLE

Scheme	How -5 (4-bit) looks	Problem
Sign-magnitude	`1101` (sign bit + magnitude 101)	two zeros (+0, -0), messy arithmetic
1's complement	`1010` (flip all bits of 5=0101)	still two zeros, needs "end-around carry"
2's complement	`1011` (flip bits, +1)	none — this is what real CPUs use

FORMULA / RULE

Rule of thumb

To get -x in 2's complement: flip every bit of x, then add 1. To go back from a negative 2's complement number to decimal: flip every bit, add 1, put a minus sign in front.

Range you must memorize

KEY POINT

For an **n-bit** signed 2's complement number:

```
Range = -2^(n-1)  to  +2^(n-1) - 1
```

So for n = 8: **-128 to +127**. Notice it's asymmetric — one extra negative number, because there's only one zero.

FORMULA / RULE

Worked numerical (GATE style)

Q: What is the range of integers representable in 12-bit 2's complement form?
A: n=12 → from -2^{11} to $+2^{11}-1$ = **−2048 to +2047**.

Addition, subtraction & overflow

NOTES

2's complement lets you **subtract by adding the negative**. Just add the bit patterns normally and discard any carry-out of the leftmost bit.

WORKED EXAMPLE

Worked numerical

Q: Using 5-bit 2's complement, compute 7 − 3.
A: 7 = `00111`, −3 = `11101` (flip 3=00011 → 11100, +1 → 11101).
Add: `00111 + 11101 = 100100` → discard the leading carry → `00100` = **4**. Correct!

COMMON TRAP

Overflow detection rule

Overflow happens only when adding two numbers of the **same sign** and the result has the **opposite sign**. Formally: overflow = carry into MSB XOR carry out of MSB. GATE loves asking "does this addition overflow?" — check the sign bits first, don't compute the whole thing if you don't have to.

WORKED EXAMPLE

Worked numerical

Q: In 4-bit 2's complement, does 5 + 4 overflow?
A: 5= `0101`, 4= `0100`. Sum = `1001` = −7 in 4-bit. Both operands positive, result negative → **overflow** (the true sum 9 is above the 4-bit signed maximum of +7, so it cannot be represented).

Floating point: IEEE 754 Core

NOTES

Fixed-point (2's complement) can't represent very large numbers or fractions efficiently. IEEE 754 splits a number into **sign, exponent, mantissa** — like scientific notation in binary.

Single precision (32-bit) layout

LAYOUT

31 S 30 E … E 23 M … M 0 M

NOTES

Sign — 1 bitExponent — 8 bitsMantissa — 23 bits

KEY POINT

Value = `(-1)^S × 1.M × 2^(E - 127)`
127 is the **bias** — it's added to the true exponent so the stored exponent is always non-negative. For double precision: 1 sign + 11 exponent (bias 1023) + 52 mantissa = 64 bits.

WORKED EXAMPLE

Worked numerical

Q: Represent 10.25 in IEEE-754 single precision.
A:
1. Binary: 10.25 = `1010.01`
2. Normalize: `1.01001 × 2³`
3. Sign = 0 (positive)
4. Exponent = 3 + 127 = 130 = `10000010`
5. Mantissa (drop leading 1, pad to 23 bits) = `01001000000000000000000`
Final: `0 10000010 01001000000000000000000`

COMMON TRAP

Special values to remember

E=0, M=0 → zero | E=0, M≠0 → denormal (subnormal) number | E=255 (all 1s), M=0 → ±infinity | E=255, M≠0 → NaN. GATE frequently asks "what does this bit pattern represent" — check E first.

Carry vs overflow, sign extension, and other codes High-yield

REFERENCE TABLE

Flag	Meaning	Test
Carry (C)	Unsigned result does not fit	Carry **out** of the MSB
Overflow (V)	Signed result does not fit	Carry **into** the MSB XOR carry **out of** the MSB

WORKED EXAMPLE

Worked example

Q: In 4 bits, add 1111 and 0001. Report carry and overflow.
A: 1111 + 0001 = 1 0000, so the result is 0000 with **carry = 1**. Unsigned, that is 15 + 1 = 16, which does not fit. Signed, it is (−1) + 1 = 0, which is correct: carry into the MSB = 1 and carry out = 1, so XOR = 0 and **overflow = 0**. The same bit pattern can set one flag and not the other.

KEY POINT

Sign extension (widening a 2's-complement number) copies the sign bit into the new high bits: 1101 (−3) in 8 bits is `11111101`, still −3. Unsigned numbers are **zero-extended** instead.

Other codes. BCD: each decimal digit in 4 bits (59 → 0101 1001). **Excess-3**: BCD digit + 3. **Gray code**: consecutive values differ in exactly one bit; for 3 bits it runs 000, 001, 011, 010, 110, 111, 101, 100, and Gray = binary XOR (binary >> 1).

2 Instruction Set & Addressing Modes

Instruction formats: how many "addresses"? Core

NOTES

An instruction = opcode (what to do) + operands (who to do it to). Older CPUs are classified by how many memory addresses an instruction can reference:

REFERENCE TABLE

Type	Example (for C = A + B)	Used in
3-address	`ADD C, A, B`	RISC-like / register machines
2-address	`MOV C, A` then `ADD C, B`	x86-style CISC
1-address (accumulator)	`LOAD A; ADD B; STORE C`	simple/old machines
0-address (stack)	`PUSH A; PUSH B; ADD; POP C`	stack machines, JVM

GATE FAVOURITE

GATE trap

Fewer addresses per instruction ⇒ more instructions needed to do the same job, but each instruction is shorter. GATE often asks you to count total instructions or total bits for a given expression across these formats.

Addressing modes — where is the actual operand? Core

NOTES

The instruction doesn't always contain the value directly. The **addressing mode** tells the CPU how to compute the **Effective Address (EA)** — where in memory/registers the real data sits.

REFERENCE TABLE

Mode	Instruction says	Effective Address / operand
Immediate	`ADD R1, #5`	operand IS 5 (no memory access)
Direct	`ADD R1, [200]`	EA = 200
Indirect	`ADD R1, [[200]]`	EA = contents of address 200 (memory holds a pointer)
Register	`ADD R1, R2`	operand is inside R2, no memory trip
Register indirect	`ADD R1, [R2]`	EA = value inside R2
Indexed	`ADD R1, [R2+X]`	EA = R2 + index register X (array access)
Base register	`ADD R1, [Base+disp]`	EA = base reg + displacement (relocatable code)
Relative	`BR [PC+disp]`	EA = PC + displacement (branches)
Auto-increment/decrement	`ADD R1, [R2]+`	EA = R2, then R2 = R2+size (array traversal)

FLOW

Indirect: Instruction → points to Address 200 → Mem[200] = Address 350 → Mem[350] = the real operand

WORKED EXAMPLE

Worked numerical

Q: An instruction uses indirect addressing with the address field = 300. Mem[300] = 500, Mem[500] = 42. What is the effective operand?
A: Indirect ⇒ EA = Mem[300] = 500. Operand = Mem[500] = **42**. (Indirect addressing costs one extra memory reference compared to direct — this is exactly why GATE loves timing questions: "if memory access = 100ns and indirect mode used, extra time = 100ns per instruction".)

WORKED EXAMPLE

Worked numerical

Q: Register R1 = 2000. Instruction: `LOAD R2, 20[R1]` using indexed addressing, and Mem[2020] = 77. Find R2 after execution.
A: EA = R1 + 20 = 2000 + 20 = 2020. R2 ← Mem[2020] = **77**.

RISC vs CISC (quick contrast, often 1 mark)

KEY POINT

RISC
Few, simple, fixed-length instructions. Mostly register-to-register. Simple addressing modes. More instructions needed, but each is fast (1 cycle typical) — pipelines well.

KEY POINT

CISC
Many complex, variable-length instructions. Instructions can directly operate on memory. Fewer instructions per program, but each may take several cycles.

Expanding opcodes Core

NOTES

A fixed-length instruction can hold more instruction types by **reusing unused opcode patterns as a prefix** for a longer opcode. Each time the opcode grows, the address fields shrink to make room.

WORKED EXAMPLE

Worked numerical

Q: A 16-bit instruction has 4-bit address fields. The machine needs 14 three-address instructions, 30 two-address instructions, and as many one-address instructions as possible. How many one-address instructions fit?
A:

```
Level 1 (3 addresses): 4-bit opcode, 3 × 4-bit addresses   →
16 patterns, use 14, 2 left
Level 2 (2 addresses): 8-bit opcode, 2 × 4-bit addresses   →
2 × 16 = 32 patterns, use 30, 2 left
Level 3 (1 address)  : 12-bit opcode, 1 × 4-bit address    →
2 × 16 = 32 patterns
```

So up to **32** one-address instructions. If only 31 were used, 1 pattern would remain: 1 × 16 = **16 zero-address** instructions with a 16-bit opcode.

Byte ordering (endianness) High-yield

NOTES

For the 32-bit value `0x12345678` stored at address 100: **big-endian** keeps the most significant byte first (100: 12, 101: 34, 102: 56, 103: 78); **little-endian** keeps the least significant byte first (100: 78, 101: 56, 102: 34, 103: 12). It matters when data is shared between machines or read byte by byte.

3 Design of the Arithmetic Logic Unit (ALU)

Adders: the building block Core

KEY POINT

Ripple Carry Adder (RCA)
n 1-bit full adders chained; carry "ripples" from LSB to MSB. Simple, but slow — the MSB must wait for every earlier carry.
Delay = $n \times \Delta$ (Δ = delay of 1 full adder).

KEY POINT

Carry Look-Ahead Adder (CLA)
Computes all carries in parallel using extra gates (generate/propagate logic), so it doesn't wait for ripples. Faster but more hardware.

WORKED EXAMPLE

Worked numerical (classic GATE pattern)
Q: A 16-bit adder is built from 4-bit carry-look-ahead adder (CLA) blocks connected by ripple carry between blocks. Each 4-bit CLA block has delay 10 ns to produce its sum and its carry-out. Find the total worst-case delay.
A: There are 16/4 = 4 blocks. Block 1 finishes in 10 ns and passes its carry-out to block 2, which then takes another 10 ns, and so on. Total delay = 4 × 10 = **40 ns**. (Contrast: if all 4 blocks used full look-ahead across blocks too, delay would be much closer to a single block's 10 ns.)

Multiplication: Booth's Algorithm Core

NOTES

Booth's algorithm multiplies signed 2's-complement numbers by scanning pairs of bits of the multiplier and deciding to **add**, **subtract**, or **do nothing** to a running partial product, then arithmetic-shift right.

KEY POINT

Rule, looking at multiplier bit Q_0 and the bit just shifted out Q_{-1}:

Q_0 Q_{-1}	Action
0 0	no operation, just shift
0 1	Add multiplicand, then shift
1 0	Subtract multiplicand, then shift
1 1	no operation, just shift

WORKED EXAMPLE

Worked numerical
Q: Multiply 3 × (−4) using Booth's algorithm (4-bit).
A: Multiplicand M = 3 = `0011`, Multiplier Q = −4 = `1100`, Q_{-1} = 0, AC = `0000`. Each step: check Q_0Q_{-1}, do the add/sub, then arithmetic-shift AC:Q:Q_{-1} right by 1.

```
Step  Q0Q-1  Action     AC after action   AC:Q:Q-1 after shift
1       0 0   none       0000              AC=0000 Q=0110 Q-1=0
2       0 0   none       0000              AC=0000 Q=0011 Q-1=0
3       1 0   AC=AC-M    1101              AC=1110 Q=1001 Q-1=1
4       1 1   none       1110              AC=1111 Q=0100 Q-1=1
```

Final product = AC:Q = `1111 0100` (8-bit 2's complement) = **−12** — matching 3 × (−4). ✔

COMMON TRAP

Why Booth's matters
It handles negative multipliers automatically and can skip strings of 0s or 1s, making multiplication faster than naive shift-and-add. GATE frequently gives you a partial trace and asks you to fill in the next AC or Q value — memorize the 4-row table above cold.

Division High-yield

NOTES

Two standard hardware algorithms, both repeat "shift, subtract divisor, check sign, restore if needed":

KEY POINT

Restoring division
Subtract divisor; if remainder goes negative, add divisor back (restore) and quotient bit = 0. Else quotient bit = 1. Simple but the extra "restore" step costs time.

KEY POINT

Non-restoring division
Never restores — instead alternates add/subtract based on the sign of the previous remainder. Same number of steps, no wasted restore cycle, so it's faster.

Carry look-ahead equations Core

NOTES

For each bit position define **generate** $G_i = A_i \cdot B_i$ (this bit makes a carry by itself) and **propagate** $P_i = A_i \oplus B_i$ (this bit passes an incoming carry on). Then $C_{i+1} = G_i + P_i \cdot C_i$ and $Sum_i = P_i \oplus C_i$. Expanding the recurrence removes the wait for ripples: $C_1 = G_0 + P_0C_0$, $C_2 = G_1 + P_1G_0 + P_1P_0C_0$, and so on. Each carry is a two-level AND-OR of G, P and C_0, so its delay does not grow with the bit position (but the gate fan-in does, which is why large adders use 4-bit blocks).

4 Design of the Control Unit

The instruction cycle in micro-operations Core

NOTES

Every instruction is really a short sequence of **register transfers**. GATE loves writing these out; you should be able to read/write them fluently.

KEY POINT

Example — the Fetch cycle (same for every instruction):

```
MAR ← PC
MBR ← Memory[MAR] , PC ← PC + 1
IR  ← MBR
```

Then Decode reads the opcode in IR, and Execute runs instruction-specific micro-ops, e.g. for `ADD R1, [500]`:

```
MAR ← 500
MBR ← Memory[MAR]
R1  ← R1 + MBR
```

Two ways to build the control unit Core

REFERENCE TABLE

	Hardwired	Microprogrammed
How signals are generated	Fixed combinational logic / FSM built from gates	A tiny stored "control program" (microprogram) in a Control Store (ROM)
Speed	Fast — pure logic delay	Slower — an extra memory read per micro-instruction
Flexibility	Hard to change — needs a re-design/re-wiring	Easy to change — just rewrite the control store contents
Best for	RISC, simple instruction sets, speed-critical CPUs	CISC, complex/large instruction sets
Cost / complexity	Complex to design, cheap to run	Simple, systematic design; needs control-store memory

Microinstruction format

NOTES

Each word in the control store is a **microinstruction**: a row of control-signal bits plus the address of the next microinstruction.

LAYOUT

Control field which signals to assert (e.g. MAR-load, ALU-add, Mem-read)
Address field address of next micro-instruction

KEY POINT

Horizontal microprogramming
One bit per control signal — wide word, no decoding needed, but uses lots of control-store space. Faster (parallel signals).

KEY POINT

Vertical microprogramming
Signals are encoded (like a mini opcode) — narrow word, needs a decoder, saves space but slightly slower.

WORKED EXAMPLE

Worked numerical

Q: A control store must hold 256 microinstructions, each needing to specify one of 8 possible micro-operations per field, using vertical microprogramming with 3 independent fields. How many bits per microinstruction (ignoring the next-address field)?
A: Each field encodes 8 choices → needs $\log_2 8 = 3$ bits. With 3 independent fields → $3 \times 3 = $ **9 bits** for control, plus $\log_2 256 = 8$ bits for the next-address field → total 17 bits/microinstruction.

GATE FAVOURITE

GATE angle to remember
Questions often give you total control store size and ask for the number of address bits ($\log_2$ of total microinstructions), or give you the number of control signals and ask you to compare horizontal (1 bit/signal) vs vertical (encoded, fewer bits) word width.

Data path: single-bus vs three-bus Core

NOTES

In a **single-bus** data path only one register can drive the bus at a time, so the ALU needs a holding register **Y** for one operand and an output register **Z** for the result. In a **three-bus** data path two buses feed the ALU and the third carries the result back, so an ALU operation takes one step.

WORKED EXAMPLE

Worked example — R3 ← R1 + R2

```
Single bus (3 control steps):
  1. R1out, Yin
  2. R2out, Add (ALU uses Y and the bus), Zin
  3. Zout, R3in
Three bus (1 step): R1 → bus A, R2 → bus B, ALU result → bus C → R3
```

The instruction fetch on a single bus also takes 3 steps: (1) PCout, MARin, Read, Select4, Add, Zin; (2) Zout, PCin, Yin, wait for memory; (3) MDRout, IRin. More buses mean fewer steps per instruction but more hardware.

5 Memory Hierarchy & Cache Mapping

The hierarchy Core

FLOW

Registers (fastest, smallest) → Cache (L1/L2/L3) → Main Memory (RAM) → Secondary storage (slowest, biggest)

NOTES

The idea: keep frequently-used data in the fast, small, expensive memory close to the CPU (locality of reference), and fall back to slower memory only on a miss.

Average Memory Access Time (AMAT) Core

KEY POINT

```
AMAT = Hit time + Miss rate × Miss penalty
```

Or in terms of hit ratio h: `Effective Access Time = h × Tc + (1 − h) × (Tc + Tm)` , where Tc = cache access time, Tm = main memory access time.

WORKED EXAMPLE

Worked numerical

Q: Cache access time = 10 ns, main memory access time = 100 ns, hit ratio = 0.9. Find effective access time.
A: EAT = 0.9×10 + 0.1×(10+100) = 9 + 11 = **20 ns**.

How an address maps into cache — 3 schemes Core

NOTES

Every physical address is split into three fields: **Tag | Set/Line index | Block offset**. The scheme decides how many places one memory block can go.

LAYOUT

Tag Set / Line index Block offset

REFERENCE TABLE

Mapping	Rule	Index field
Direct mapped	Each memory block → exactly ONE cache line	present — determines the single line
Fully associative	A block can go in ANY cache line	none — compare tag against all lines
Set-associative (n-way)	A block maps to ONE set, but can sit in any of the n lines in that set	present — picks the set, then search n lines

WORKED EXAMPLE

Worked numerical (very common GATE pattern)

Q: Main memory = 4 GB (32-bit address), cache = 64 KB, block size = 32 bytes, direct-mapped. Find tag, index, and offset bit widths.
A:
Offset bits = $\log_2$(block size) = $\log_2 32 =$ **5 bits**
Number of cache lines = cache size / block size = 65536/32 = 2048 lines → index bits = $\log_2 2048 =$ **11 bits**
Tag bits = total address bits − index − offset = 32 − 11 − 5 = **16 bits**

WORKED EXAMPLE

Worked numerical — 4-way set associative

Q: Same cache (64 KB, 32-byte blocks) but organized as 4-way set-associative. Find the index bits.
A: Number of sets = lines / ways = 2048 / 4 = 512 sets → index bits = $\log_2 512$ = **9 bits**. Tag bits become 32 − 9 − 5 = **18 bits** (bigger tag because fewer sets to search).

GATE FAVOURITE

Replacement & write policy — quick recall

When a set is full, a block must be evicted: **LRU** (least recently used) is the GATE favorite, also FIFO / random. Write policy: **write-through** (update memory immediately, simple, slower) vs **write-back** (update memory only on eviction, needs a "dirty bit", faster).

Write misses and cache miss types Core

REFERENCE TABLE

Policy pair	On a write hit	On a write miss
Write-through + **no-write-allocate**	Update cache and memory	Write memory only; the block is not brought into the cache
Write-back + **write-allocate**	Update the cache and set the dirty bit	Fetch the block into the cache, then write it there

NOTES

A write-back cache writes memory only when a **dirty** block is evicted, which saves bus traffic, but a replacement may then cost an extra memory write. **Miss types (the 3 Cs): compulsory** (first access to a block), **capacity** (the working set exceeds the cache) and **conflict** (too many blocks map to the same set; a fully associative cache has none).

Multi-level cache access time Core

NOTES

$AMAT = t_1 + m_1 \cdot (t_2 + m_2 \cdot t_mem)$ where m_i is the miss rate of level i (local miss rate) and t_i its access time.

WORKED EXAMPLE

Worked numerical

Q: L1: 1 ns, miss rate 10%. L2: 10 ns, local miss rate 20%. Memory: 100 ns. Find the average access time.

A: AMAT = 1 + 0.1 × (10 + 0.2 × 100) = 1 + 0.1 × 30 = **4 ns**.

Memory interleaving and memory types High-yield

NOTES

With **m-way low-order interleaving**, consecutive addresses fall in different banks (bank = address mod m), so m accesses can overlap. **Example:** 4 banks, 100 ns per bank access: fetching 8 consecutive words takes 2 × 100 = **200 ns** (two rounds of 4 parallel accesses), against 800 ns from a single bank, assuming no bank conflicts.

REFERENCE TABLE

	SRAM	DRAM
Cell	Flip-flop (about 6 transistors)	1 transistor + 1 capacitor
Refresh	Not needed	**Needed** (charge leaks)
Speed / cost / density	Faster, costlier, less dense	Slower, cheaper, denser
Typical use	Cache	Main memory

Memory interfacing — chips, decoding & expansion Core

1. A memory chip, as the CPU sees it

NOTES

A chip of `N × w` (N words, w bits per word) has **$\log_2$N address lines**, **w data lines**, a **chip select (CS)** and read/write controls (RD/WR or OE/WE). CS is usually **active-low**: with CS inactive the chip ignores the address and its data lines float (tri-state), so many chips can share one data bus.

REFERENCE TABLE

Chip	Address lines	Data lines	Capacity
1K × 8	10	8	1 KB
16K × 1	14	1	2 KB
4M × 8	22	8	4 MB

2. Address decoding & chip select

FLOW

CPU address A(k−1) … A(m) (high bits) → Decoder → CS of one chip / chip group

FLOW

CPU address A(m−1) … A0 (low bits) → Address pins of every chip in parallel

KEY POINT

Rule. Low bits (as many as the chip has address lines) go to **all** chips and pick the word *inside* a chip. High bits go to a **decoder** (or logic) that raises exactly one CS and picks *which* chip. The decoder is enabled only for the address bits that must have a fixed value.

COMMON TRAP

Trap — partial decoding

If some high address bits are **not** used in the decoder, they become don't-cares and the same chip answers at several address ranges (aliasing / "mirroring"). Full decoding uses every high bit and gives a unique address for every location.

3. Expansion — two independent directions

REFERENCE TABLE

Expansion	What grows	How the chips connect
Word-size (width)	Bits per word	Chips side by side: **same address, same CS**; data lines are concatenated (e.g. two 8-bit chips → 16-bit word)
Capacity (depth)	Number of words	Chips stacked: low bits to all, **high bits → decoder → separate CS**; data lines are shared on the bus

FORMULA / RULE

```
Chips needed = (N × W) / (n × w)          memory N × W built from chips n × w
Columns (word-size) = W / w               Rows (capacity) = N / n
Decoder inputs = log₂(Rows)               Rows = number of CS lines
```

4. Representative problems

NOTE

Problem 1 — expansion in both directions

Q: Build a **4K × 16** memory from **1K × 4** chips. How many chips, how are they arranged, and how are the address bits used?

A: Chips = (4096 × 16)/(1024 × 4) = **16**. Columns = 16/4 = 4 chips side by side give the 16-bit word; rows = 4096/1024 = 4 groups give the depth. The 12-bit address splits as: **A9–A0** (10 bits) to every chip; **A11–A10** (2 bits) to a **2-to-4 decoder** whose four outputs are the CS of the four rows.

FORMULA / RULE

Problem 2 — address ranges from a decoder

Q: A CPU has a 16-bit address bus. Four **2K × 8** chips are placed contiguously from 0000H, with A15–A13 required to be 000. Give each chip's range.

A: Each chip has 11 address lines (A10–A0) and covers 800H bytes. A15–A13 = 000 enables a 2-to-4 decoder on **A12, A11**:

```
Chip 0 (A12A11 = 00): 0000H – 07FFH
Chip 1 (A12A11 = 01): 0800H – 0FFFH
Chip 2 (A12A11 = 10): 1000H – 17FFH
Chip 3 (A12A11 = 11): 1800H – 1FFFH       total 8 KB
```

Because A15–A13 are decoded, the chips do not alias elsewhere in the 64 KB space. If A15–A13 were ignored, the same 8 KB would repeat 8 times (partial decoding).

NOTE

Problem 3 — decoder sizing

Q: A 16-bit address bus drives **64 KB** of memory made of **8K × 8** chips. How many chips and which decoder?

A: Chips = 64K/8K = **8** (all in one column, as the word is already 8 bits). Each chip needs 13 address lines (A12–A0); the remaining 3 bits **A15–A13** feed a **3-to-8 decoder** whose 8 outputs are the CS lines. Chip i occupies i × 2000H to i × 2000H + 1FFFH.

COMMON TRAP

Byte-addressable memory on a wider bus

On a byte-addressable machine with a 32-bit data bus, the memory is built as 4 byte-wide banks. The 2 lowest address bits (A1–A0) choose the byte within the word and are **not** sent to the chips; the chips see A(k−1)–A2. Read the question to see whether the "word" is the byte or the bus width.

Virtual memory and the TLB High-yield

NOTES

A virtual address splits into **page number | offset**. The page table maps page → frame. Page-table size = (number of virtual pages) × (entry size): a 32-bit virtual address with 4 KB pages has 2^{20} pages, so with 4-byte entries the table is 2^{20} × 4 B = **4 MB**. A **TLB** caches recent translations so most accesses skip the page-table lookup in memory. With a single-level page table:

WORKED EXAMPLE

Worked numerical

Q: TLB access 10 ns, memory access 100 ns, TLB hit ratio 90%. Find the effective access time.

A: A hit costs 10 + 100 = 110 ns. A miss costs 10 + 100 (page-table read) + 100 (data) = 210 ns. EMAT = 0.9 × 110 + 0.1 × 210 = 99 + 21 = **120 ns**.

Secondary storage — disk access time Extension

NOTES

Access time = seek time + rotational latency + transfer time. Average rotational latency is **half a rotation**; transfer time = (bytes wanted / bytes per track) × rotation time.

WORKED EXAMPLE

Worked numerical

Q: Seek time 4 ms, 7200 rpm, 400 sectors per track. Find the time to read one sector.

A: One rotation = 60/7200 s = 8.33 ms, so the average latency is 4.17 ms and one sector passes under the head in 8.33/400 = 0.021 ms. Total = 4 + 4.17 + 0.02 ≈ **8.19 ms**.

6 I/O Interface: Interrupts & DMA

KEY POINT

Programmed I/O

CPU repeatedly polls the device's status flag in a busy-wait loop. Simple, but wastes CPU cycles.

KEY POINT

Interrupt-driven I/O

CPU carries on with other work; the device raises an interrupt signal only when it's ready. CPU saves state, runs the ISR (interrupt service routine), then resumes. Saves CPU time vs polling, but each transfer still passes through the CPU register-by-register.

KEY POINT

DMA (Direct Memory Access)

A separate DMA controller moves an entire block directly between device and memory, without the CPU touching each word. CPU just sets up (starting address, count, direction) then is interrupted once at the very end. Best for bulk transfers (disk, network).

Cycle stealing

NOTES

While a DMA transfer is in progress, the DMA controller "steals" memory bus cycles from the CPU one at a time, so both can (almost) work concurrently — this is why DMA is much more efficient than interrupt-driven I/O for large transfers.

WORKED EXAMPLE

Worked numerical

Q: A device transfers data at 1 MB/s. Each interrupt-driven transfer moves 4 bytes and costs the CPU 100 cycles of overhead (context switch + ISR) at 1 GHz. Compare CPU overhead per second against a DMA scheme that interrupts only once per 4 KB block.

A:

Interrupt-driven: transfers/sec = 1,000,000/4 = 250,000 interrupts/sec → overhead = 250,000 × 100 cycles = 25,000,000 cycles/sec = **2.5% of a 1 GHz CPU** just on overhead.

DMA: interrupts/sec = 1,000,000/4096 ≈ 244 → overhead ≈ 24,400 cycles/sec — **roughly 1000× less CPU overhead.**

GATE FAVOURITE

GATE angle

Typical questions: given interrupt latency & instruction execution time, compute total time lost to interrupts; or compare polling vs interrupt vs DMA overhead like above. Also know: **vectored** interrupts (device ID tells CPU where its ISR is) vs **non-vectored** (fixed single ISR address, must poll to find the source).

Interrupt priority: daisy chain and nesting Core

NOTES

When several devices share one interrupt line, the CPU needs a priority scheme. **Daisy chain:** the interrupt-acknowledge signal passes serially from device to device, so the device **nearest the CPU has the highest priority**. **Parallel priority** uses a separate request line per device, a mask register and a priority encoder; it is faster but needs more hardware. With **nested interrupts** a higher-priority request can interrupt a running ISR while lower-priority ones stay masked. The CPU finishes the current instruction before it responds to an interrupt.

Memory-mapped vs isolated I/O Core

REFERENCE TABLE

	Memory-mapped I/O	Isolated (port-mapped) I/O
Address space	Device registers share the memory address space	Separate I/O address space
Instructions	Ordinary load/store	Special IN / OUT
Cost	Uses up part of the memory address range	Full memory range kept; needs an extra control signal

DMA transfer modes Core

REFERENCE TABLE

Mode	How it uses the bus	CPU effect
Burst (block)	Controller keeps the bus for the whole block	CPU is blocked from memory for the entire burst
Cycle stealing	Takes the bus for one word at a time	CPU slowed slightly
Transparent	Uses the bus only when the CPU is not using it	No slowdown, but the transfer may be slower

WORKED EXAMPLE

Worked numerical

Q: A device transfers 4 MB/s in 4-byte words using cycle stealing. Each stolen memory cycle lasts 100 ns. What fraction of the memory bus time does DMA use?

A: 4 MB/s ÷ 4 B = 1 million words per second. Each steals 100 ns: 10^6 × 100 ns = 0.1 s per second = **10%** of the bus time, so the CPU keeps the other 90%.

7 Instruction Pipelining & Hazards

The classic 5-stage pipeline Core

KEY POINT

`IF` Instruction Fetch → `ID` Instruction Decode → `EX` Execute → `MEM` Memory access → `WB` Write back

REFERENCE TABLE

Cycle →	1	2	3	4	5	6
Instr 1	IF	ID	EX	MEM	WB	
Instr 2		IF	ID	EX	MEM	WB

NOTES

Without pipelining each instruction takes 5 cycles start-to-finish; with pipelining, once the pipe is full, one instruction **completes every cycle**.

Speedup formula

KEY POINT

```
Time (non-pipelined) = n × k
Time (pipelined)     = k + (n − 1)
Speedup = (n × k) / (k + n − 1)
```

n = number of instructions, k = number of pipeline stages. As n→∞, speedup → k.

WORKED EXAMPLE

Worked numerical

Q: A 5-stage pipeline, clock cycle = 2 ns, runs 100 instructions with no stalls. Find total time and speedup vs non-pipelined execution.
A: Pipelined cycles = k + n − 1 = 5 + 99 = 104 → time = 104 × 2 = **208 ns**.
Non-pipelined time = n × k × cycle = 100 × 5 × 2 = 1000 ns.
Speedup = 1000 / 208 ≈ **4.8×** (close to the ideal max of 5).

Hazards — the reason pipelines stall Core

REFERENCE TABLE

Hazard	Cause	Fix
Structural	Two instructions need the same hardware resource (e.g. one memory port for both instruction fetch and data access) at once	Duplicate hardware (separate I-cache/D-cache), or stall
Data	An instruction needs a result that a previous, still-unfinished instruction hasn't produced yet	Forwarding/bypassing (feed result directly from EX/MEM stage), or stall
Control (branch)	The next instruction to fetch isn't known until a branch is resolved (usually in EX/MEM stage)	Branch prediction, delayed branch, or stall until resolved

WORKED EXAMPLE

Worked numerical — data hazard with stalls

Q: `ADD R1, R2, R3` is immediately followed by `SUB R4, R1, R5` in a 5-stage pipeline with no forwarding. R1 is written in WB (cycle 5 of instr 1) but needed by SUB's EX stage. How many stall (bubble) cycles are needed?
A: Without forwarding, SUB's ID stage (where operands are read) must happen no earlier than the cycle right after instr 1's WB. Instr 1: IF ID EX MEM WB (cycles 1-5). Instr 2 naturally would be ID at cycle 3 — but it needs the value only ready after cycle 5. So ID must be delayed to cycle 6 → **3 stall cycles** inserted. (With full forwarding from EX/MEM to the next instruction's EX, this typically drops to 0–1 stalls.)

GATE FAVOURITE

GATE angle

Questions typically give a sequence of dependent instructions and a pipeline diagram, then ask: total cycles with stalls, or how many bubbles a specific forwarding path removes, or the effect of a branch misprediction (stall cycles = number of stages before branch resolves).

Data hazard types: RAW, WAR, WAW Core

REFERENCE TABLE

Hazard	Meaning	Occurs in a simple in-order 5-stage pipeline?
RAW (true dependency)	A later instruction reads a register before an earlier one has written it	**Yes** — the main cause of stalls
WAR (anti-dependency)	A later instruction writes a register before an earlier one has read it	No — reads happen early, writes late
WAW (output dependency)	Two instructions write the same register; the writes could complete out of order	No — writes stay in order

NOTES

WAR and WAW appear with out-of-order execution or instructions of different latency, and are removed by **register renaming**. With **forwarding**, an ALU result flows from EX/MEM straight into the next instruction's EX stage, removing the stall for back-to-back ALU instructions. A **load followed by a use of the loaded value** still costs **1 stall cycle** even with forwarding, because the data is only available after MEM.

Branch penalty and prediction Core

NOTES

CPI = 1 + (branch frequency) × (fraction mispredicted or stalled) × (penalty in cycles). With 20% branches and a 3-cycle penalty, always stalling gives CPI = 1 + 0.2 × 3 = **1.6**; with a predictor that is wrong 40% of the time, CPI = 1 + 0.2 × 0.4 × 3 = **1.24**. Techniques: **static** prediction (always not-taken, or backward-taken/forward-not-taken), **delayed branch** (the instruction in the delay slot always executes), and **dynamic** prediction with a branch history table. A **2-bit saturating counter** needs two wrong predictions in a row to change its mind, so a loop branch mispredicts only once per loop exit.

Pipeline with unequal stages Core

NOTES

The clock must fit the **slowest stage plus the latch delay**. A non-pipelined machine takes the **sum** of the stage delays per instruction.

WORKED EXAMPLE

Worked numerical

Q: Four stages take 5, 10, 8 and 7 ns, and each pipeline latch adds 1 ns. Find the speedup for 100 instructions.
A: Clock = 10 + 1 = 11 ns. Pipelined time = (4 + 99) × 11 = 1133 ns. Non-pipelined time = (5 + 10 + 8 + 7) × 100 = 3000 ns. Speedup = 3000 / 1133 ≈ **2.65**, far below 4 because the slow stage sets the clock for everyone.

CPU performance: CPI, MIPS and Amdahl's law High-yield

NOTES

CPU time = instruction count × CPI × clock cycle time; **MIPS = clock rate / (CPI × 10^6)**. For a mix, CPI is the frequency-weighted average of the per-class cycle counts. **Amdahl's law:** if a fraction f of the time is sped up by a factor s, `Speedup = 1 / ((1 − f) + f/s)`, and the limit as s → ∞ is 1/(1 − f).

WORKED EXAMPLE

Worked numerical

Q: A 1 GHz CPU runs a mix of 50% ALU (1 cycle), 20% loads (2), 10% stores (2) and 20% branches (3). Find CPI, MIPS and the time for 10^9 instructions. Separately, if 80% of a program is sped up 4×, what is the overall speedup?
A: CPI = 0.5·1 + 0.2·2 + 0.1·2 + 0.2·3 = 0.5 + 0.4 + 0.2 + 0.6 = **1.7**. MIPS = 1000 / 1.7 ≈ **588**. Time = 10^9 × 1.7 × 1 ns = **1.7 s**. Amdahl: 1 / (0.2 + 0.8/4) = 1 / 0.4 = **2.5**, and even an infinite speedup of that part could give at most 1/0.2 = 5.

Flynn's taxonomy and instruction-level parallelism Extension

REFERENCE TABLE

Class	Instruction streams	Data streams	Example
SISD	1	1	Classic uniprocessor
SIMD	1	many	Vector/GPU-style units
MISD	many	1	Rare (fault-tolerant designs)
MIMD	many	many	Multicore, clusters

NOTES

A **superscalar** CPU issues several instructions per cycle (CPI can fall below 1); a **VLIW** machine has the compiler pack independent operations into one long instruction word.

8 Problem → Standard Method Playbook

METHOD

Instruction Encoding & Datapath

Problem pattern	Standard method	Key check
Instruction length / opcode fields	Count total encoding bits and required operand/register/address bits; remainder is opcode/function space.	Check whether the question uses fixed or variable/expanding opcode.
Expanding opcode	Allocate short opcodes first; each longer opcode consumes a prefix branch of the code space.	Never count all bit patterns independently when prefixes are constrained.
Single-bus datapath	Trace one transfer per bus cycle; identify source register → bus → destination and ALU control.	Two registers cannot normally drive the same single bus simultaneously.
Multi-bus datapath	Trace simultaneous source reads and destination write-back using separate buses.	More buses can reduce cycles but add hardware.
Endianness	Split a multi-byte value into bytes and place the least/most significant byte at the lowest address according to little/big endian.	Endianness changes byte order, not bit order within a byte.

METHOD

Cache & Memory

Pattern	Standard method	Formula / trap
Cache address split	Offset bits=$\log_2$(block size); number of lines/sets determines index bits; remaining bits are tag.	For S-way set associative cache, sets=lines/S.
AMAT	AMAT=hit time + miss rate×miss penalty.	For multi-level cache, expand recursively with conditional probabilities.
Write policy	Distinguish write-through/write-back and write-allocate/no-write-allocate.	Write-back updates memory on eviction; write-through updates memory on write.
Memory interleaving	Distribute consecutive words across banks; determine bank number from address modulo number of banks.	Check whether the question assumes low-order or high-order interleaving.
Secondary storage	Use block/sector geometry and seek/rotational/transfer components as specified.	Don't invent a seek component if the problem gives only transfer time.

METHOD

Pipeline & Performance

Pattern	Method	Trap
Pipeline speed up	Compare non-pipelined time with pipeline fill + steady-state cycles.	For n instructions and k stages, ideal cycles=n+k−1.
Hazard counting	Identify RAW/WAR/WAW and structural/control hazards; then apply forwarding/stalls/branch penalty stated in the question.	Forwarding does not remove every RAW hazard.
CPI/performance	CPU time = Instruction Count × CPI × Clock Cycle Time.	Weighted CPI if instruction classes have different CPIs.
Amdahl's law	Speedup=1/[(1−f)+f/s], where f is fraction improved and s is speedup of that fraction.	Speedup applies to execution-time fraction, not necessarily instruction fraction.

METHOD

I/O & Interrupts

Pattern	Standard method	Key distinction
Programmed I/O	CPU polls device status and transfers data.	CPU remains involved in waiting.
Interrupt-driven I/O	Device interrupts CPU when service is needed; CPU runs ISR.	Know interrupt latency and priority assumptions.
DMA	DMA controller transfers blocks between device and memory with limited CPU intervention.	Distinguish DMA setup time from CPU transfer time.
Priority / daisy chain	Apply the stated priority ordering to simultaneous requests; daisy chaining gives priority according to physical chain position.	Priority is an ordering, not a bandwidth calculation.
Memory-mapped vs isolated I/O	Check whether I/O registers share the memory address space or use a separate I/O address space.	This affects instruction/address-space assumptions.

METHOD

🧮 Representative GATE-style Worked Problems

Problem 1 · Expanding opcode

An ISA has 8-bit instructions. A 4-bit opcode identifies 8 instructions using a prefix scheme. How do you reason about additional opcode space?

Method: Draw the prefix tree: each unused prefix branch can be expanded only if it does not collide with an existing complete opcode.

Takeaway: Count available prefix leaves, not raw 2^n patterns.

Problem 2 · Datapath

A single-bus datapath must execute R3 ← R1+R2. What transfers are required?

Method: Place R1 on the bus → latch into ALU input A; place R2 on bus → ALU adds; place result on bus → load R3.

Takeaway: A single bus generally serializes source transfers.

Problem 3 · Amdahl

40% of execution time is improved by a factor of 4. Find overall speedup.

Method: $S=1/[(1-0.4)+0.4/4]$.

Answer: $1/0.7 = 10/7 \approx 1.43\times$.

Problem 4 · Cache

A cache has 64 lines and is 4-way set associative. How many sets?

Method: Sets=64/4.

Answer: 16 sets; therefore index bits=$\log_2 16=4$.

Problem 5 · Interleaving

Memory has 4 banks with low-order interleaving. Which bank receives word address A?

Method: bank=A mod 4.

Trap: If high-order interleaving is specified, the mapping is different.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Memory interfacing:** chips needed = (N×W)/(n×w). Word-size expansion = chips in parallel (same address and CS, data lines concatenated); capacity expansion = chips stacked (high address bits → decoder → CS).
- Effective address questions require careful attention to base/index/offset semantics and word vs byte addressing.
- Address bits = tag + set/index bits + block-offset bits; offset bits = $\log_2$(block size in bytes).
- Pipeline speedup is limited by stage imbalance, register overhead, stalls, and pipeline fill/drain effects.
- **Expanding opcode:** each level multiplies leftover patterns by $2^{\text{address bits freed}}$. Carry = carry out of the MSB (unsigned); overflow = carry into MSB XOR carry out (signed).
- Multi-level AMAT = t1 + m1·(t2 + m2·t_mem). TLB EMAT (single level) = h·(t_tlb + t_mem) + (1−h)·(t_tlb + 2·t_mem). Disk access = seek + half-rotation + transfer.
- Pipeline clock = slowest stage + latch delay. CPI = 1 + branch freq × mispredict rate × penalty. CPU time = IC × CPI × cycle time. Amdahl: 1/((1−f)+f/s).

LAST-MINUTE · MUST REMEMBER

- Low address bits go to every chip (one chip's worth: $\log_2 n$ bits); the remaining high bits feed a decoder whose outputs are the chip selects. CS is usually active-low.
- Two's complement represents signed integers; the most significant bit acts as the sign bit.
- Addressing modes determine how an instruction obtains its operand/effective address; distinguish immediate, direct, indirect, register, and indexed forms.
- **Cache mapping types:** direct-mapped, fully associative, and set-associative.
- DMA transfers blocks between I/O and memory with limited CPU involvement; interrupts notify the CPU of events.
- **Pipeline hazards:** structural, data, and control hazards.
- Know branch penalties, forwarding, stalling, and branch prediction at the level required for pipeline traces.
- CNF can be built as a conjunction of clauses; DNF as a disjunction of terms. Use truth-table rows when algebra becomes error-prone.
- **Cayley–Hamilton:** every square matrix satisfies its own characteristic polynomial; use it to reduce high powers of A.
- For a connected undirected graph with n−1 edges, the graph is a tree; every tree edge is a bridge.
- Convergence-test choice is pattern-driven: ratio/root for factorials and powers; comparison/integral for rational-like terms.
- Instruction encoding problems are bit-budget problems: total instruction bits = opcode/function bits + operand/address bits.
- For an S-way set-associative cache, number of sets = total cache lines / S.
- Single-bus datapaths serialize bus transfers; multi-bus datapaths can read multiple sources concurrently.
- Low-order interleaving maps word address A to bank A mod number_of_banks; high-order interleaving uses higher address bits.
- Memory-mapped I/O shares the memory address space; isolated I/O uses a separate I/O address space.
- Write-back + write-allocate vs write-through + no-write-allocate. Miss types: compulsory, capacity, conflict (none in a fully associative cache).
- **Daisy chain:** nearest device has highest priority. DMA modes: burst blocks the CPU, cycle stealing takes one word at a time, transparent uses idle bus cycles. Memory-mapped I/O uses ordinary load/store.
- Single-bus data path needs Y and Z registers (3 steps for R3 ← R1 + R2); three-bus does it in 1 step. Endianness: big-endian stores the most significant byte first.

LAST-MINUTE · COMMON TRAPS

- Partial address decoding leaves unused high bits as don't-cares, so one chip answers at several address ranges (aliasing). Full decoding gives each location a unique address.
- Overflow in signed fixed-width arithmetic is distinct from carry out of the most significant bit.
- AMAT depends on hit time and miss penalty; do not treat cache hit rate as the only performance factor.
- Zero covariance does not generally imply independence. For independence, factor the joint distribution into marginals.
- Endianness changes byte order across addresses; it does not reverse the bit order inside each byte.
- Forwarding does not eliminate every pipeline stall; a load-use RAW hazard may still require a bubble depending on the pipeline.
- Only RAW hazards stall a simple in-order pipeline; WAR and WAW need out-of-order execution. A load-use pair still costs 1 stall even with forwarding.

LAST-MINUTE · GATE FAVOURITES

- Data hazards include RAW, WAR, and WAW; in a simple in-order pipeline, RAW is the central true dependency hazard.
- For a finite set, equivalence relations correspond exactly to partitions of the set.
- **For binomial X~Bin(n,p):** E=np and Var=np(1−p). For Poisson(λ): E=Var=λ.
- Use complement for 'at least one', inclusion–exclusion for overlapping cases, and pigeonhole for guaranteed repetition.
- Expanding opcode questions are best solved with a prefix-tree/code-space view, not by blindly counting all bit patterns.
- Amdahl speedup = 1/[(1−f)+f/s], where f is the original execution-time fraction improved by factor s.
- DMA transfers blocks between an I/O device and memory with much less CPU involvement than programmed I/O.